

Welcome to SEAS Online at George Washington University

Class will begin shortly

Audio: To eliminate background noise, please be sure your audio is muted. To speak, please click the hand icon at the bottom of your screen (**Raise Hand**). When instructor calls on you, click microphone icon to unmute. When you've finished speaking, ***be sure to mute yourself again.***

Chat: Please type your questions in Chat.

Recordings: As part of the educational support for students, we provide downloadable recordings of each class session to be used exclusively by registered students in that particular class. **Releasing these recordings is strictly prohibited.**

SEAS 8515 – Lecture 5

Data Transformation

Walid Hassan, D.Eng., M.B.A.

Spark vs. Python DataFrame

Spark DataFrame	Pandas DataFrame
Spark DataFrame supports parallelization.	Pandas DataFrame does not support parallelization.
Spark DataFrame has Multiple Nodes.	Pandas DataFrame has a Single Node.
It follows Lazy Execution which means that a task is not executed until an action is performed.	It follows Eager Execution, which means task is executed immediately.
Spark DataFrame is Immutable.	Pandas DataFrame is Mutable.
Complex operations are difficult to perform as compared to Pandas DataFrame.	Complex operations are easier to perform as compared to Spark DataFrame.
Spark DataFrame is distributed and hence processing in the Spark DataFrame is faster for a large amount of data.	Pandas DataFrame is not distributed and hence processing in the Pandas DataFrame will be slower for a large amount of data.
<code>sparkDataFrame.count()</code> returns the number of rows.	<code>pandasDataFrame.count()</code> returns the number of non NA/null observations for each column.
Spark DataFrames are excellent for building a scalable application.	Pandas DataFrames can't be used to build a scalable application.
Spark DataFrame assures fault tolerance.	Pandas DataFrame does not assure fault tolerance. We need to implement our own framework to assure it.

Src: <https://www.geeksforgeeks.org/difference-between-spark-dataframe-and-pandas-dataframe/>

Spark vs. Python DataFrame

PySpark DataFrames and Pandas DataFrames look similar, they are not interchangeable, and some functions available in Pandas are not available in Spark, especially for row-level and in-memory style operations.

Use Spark DataFrames if:

- Data is larger than memory ($> \sim 1\text{--}2\text{GB}$ on local, or TB scale)
- You need to scale out processing across a cluster
- You're running on distributed storage (e.g., S3, ADLS, HDFS)
- You plan to use streaming, Delta Lake, or ML pipelines across large datasets
- You're preparing data for use in data lakes, ETL, or BI systems
- You want resilience (i.e., Spark jobs don't crash on partial failures)

Spark vs. Python DataFrame

```
pandas_df = spark_df.toPandas()
```

Spark will:

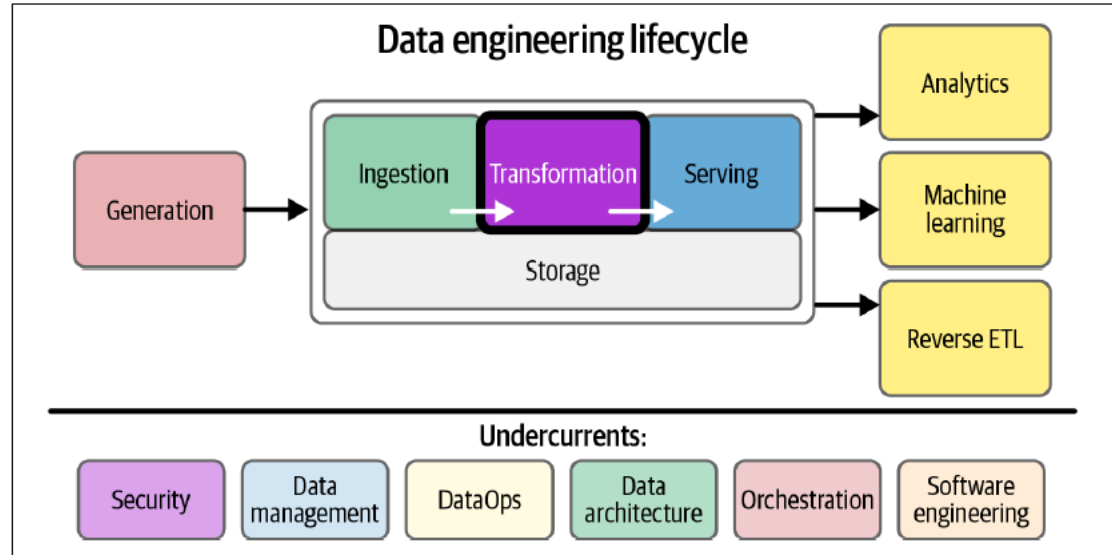
- Collect the entire distributed DataFrame (i.e., all partitions) to the driver node.
- Convert it into a single in-memory Pandas DataFrame.
- Return it as a `pandas.DataFrame` object, fully loaded into local memory.

```
spark.createDataFrame(pandas_df)
```

- Spark driver receives the Pandas DataFrame (already local).
- It serializes and distributes the data to executor nodes.
- Spark creates a DataFrame with inferred or user-provided schema.
- The new Spark DataFrame can now run in distributed fashion.

Data Engineering Lifecycle

- Generation
- Storage
- Ingestion
- Transformation
- Serving



Transformations allow us to create value from data

Src: Fundamentals of Data Engineering (Reis & Housley)

© Walid Hassan, M.B.A, D.Eng.

What is a Query

- A query allows you to retrieve and act on data. When a query retrieves data, it is issuing a request to read a pattern of records.
- This is the R (read) in CRUD. You might issue a query that gets all records from a table foo, such as `SELECT * FROM foo`. Or, you might apply a predicate (logical condition) to filter your data by retrieving only records where the id is 1, using the SQL query `SELECT * FROM foo WHERE id=1`.
- Many databases allow you to create, update, and delete data. These are the CUD in CRUD; your query will either create, mutate, or destroy existing records.

Data Definition Language (DDL)

- You first need to create the database objects before adding data. You'll use data definition language (DDL) commands to perform operations on database objects, such as the database itself, schemas, tables, or users; DDL defines the state of objects in your database.
- Data engineers use common SQL DDL expressions: CREATE, DROP, and UPDATE. For example, you can create a database by using the DDL expression `CREATE DATABASE bar`. After that, you can also create new tables (`CREATE table bar_table`) or delete a table (`DROP table bar_table`).

Src: Fundamentals of Data Engineering (Reis & Housley)

© Walid Hassan, M.B.A, D.Eng.

Data Manipulation Language (DML)

- After using DDL to define database objects, data manipulation language (DML) is used to add and alter data within these objects.
- Some common DML commands are :
 - SELECT
 - INSERT
 - UPDATE
 - DELETE
 - COPY
 - MERGE

Data Manipulation Language (DML)

SELECT:

- Query and retrieve data from a table or DataFrame.
- `SELECT * FROM employees WHERE department = 'HR';`

INSERT

- Insert new records into a table.
- `INSERT INTO employees VALUES (1, 'Alice', 'HR');`

UPDATE

- Update existing records in a table.
- `UPDATE employees SET department = 'Finance' WHERE id = 1;`

Src: Fundamentals of Data Engineering (Reis & Housley)

Data Manipulation Language (DML)

DELETE

- Delete records from a table.
- `DELETE FROM employees WHERE id = 1;`

COPY:

- *COPY INTO* command in Databricks to load data
- *COPY INTO* `delta.`/mnt/data/employees``
- *FROM* `'/mnt/data/raw/employees.csv'`
- *FILEFORMAT* = `'csv'`
- *FORMAT_OPTIONS* (`'header' = 'true'`);

Src: Fundamentals of Data Engineering (Reis & Housley)

© Walid Hassan, M.B.A, D.Eng.

Data Manipulation Language (DML)

MERGE:

- MERGE INTO allows you to upsert (update or insert) data based on conditions.
- It's similar to the SQL MERGE statement in relational databases.
- Supported in Delta Lake tables, which provide ACID transactions and versioning in Spark.

Data Manipulation Language (DML)

MERGE INTO target_table AS target

USING source_table AS source

ON target.id = source.id

WHEN MATCHED AND <condition> THEN

UPDATE SET target.column1 = source.column1

WHEN MATCHED AND <condition> THEN

DELETE

WHEN NOT MATCHED THEN

INSERT (id, column1, column2) VALUES (source.id, source.column1, source.column2)

Src: Fundamentals of Data Engineering (Reis & Housley)

© Walid Hassan, M.B.A, D.Eng.

Data Manipulation Language (DML)

ON Clause:

- Specifies the condition to match rows between the target and source tables.
- Example: ON target.id = source.id

WHEN MATCHED:

- Actions to perform when the ON condition matches rows in both the target and source tables. Operations:
 - UPDATE: Update columns in the target table.
 - DELETE: Delete rows from the target table

Data Manipulation Language (DML)

WHEN NOT MATCHED:

- Action to perform when the ON condition does not find a match in the target table.
- Operation:
 - INSERT: Insert new rows into the target table.

Data Manipulation Language (DML)

Recall What we did before for the department example

```
delta_table.alias("existing").merge(  
    new_df.alias("updates"),  
    "existing.Name = updates.Name"  
).whenMatchedUpdate(set={"Age": "updates.Age", "Department": "updates.Department"}) \  
    .whenNotMatchedInsert(values={"Name": "updates.Name", "Age": "updates.Age", "Department":  
    "updates.Department"}) \  
    .execute()
```

- whenMatchedUpdate (set=...etc)
- whenMatchedUpdateAll: Updates all columns in the target table with corresponding columns from the source table when a match is found.
- whenMatchedDelete: Deletes the target table's records when a match is found and a specified condition is met. E.g. whenMatchedDelete(condition="updates.Age > 65")
- whenNotMatchedInsert: Inserts new records into the target table when no match is found.
- whenNotMatchedInsertAll: Inserts all columns from the source into the target table when no match is found

Src: Fundamentals of Data Engineering (Reis & Housley)

© Walid Hassan, M.B.A, D.Eng.

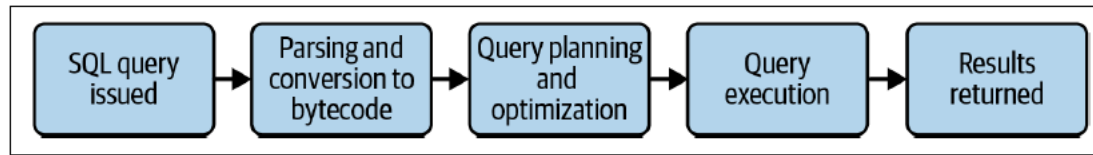
Data Control Language (DCL)

- Data control language (DCL) allows you to control access to the database objects or the data by using SQL commands such as GRANT, DENY, and REVOKE.
- Let's walk through a brief example using DCL commands. A new data scientist named Sarah joins your company, and she needs read-only access to a database called data_science_db. You give Sarah access to this database by using the following DCL command:
 - GRANT SELECT ON data_science_db TO user_name Sarah;

Transaction Control Language(TCL)

- As its name suggests, transaction control language (TCL) supports commands that control the details of transactions. With TCL, we can define commit checkpoints, conditions when actions will be rolled back, and more. Two common TCL commands
- include COMMIT and ROLLBACK.

Life of a Query



- 1. The database engine compiles the SQL, parsing the code to check for proper semantics and ensuring that the database objects referenced exist and that the current user has the appropriate access to these objects.
- 2. The SQL code is converted into bytecode. This bytecode expresses the steps that must be executed on the database engine in an efficient, machine-readable format.
- 3. The database's query optimizer analyzes the bytecode to determine how to execute the query, reordering and refactoring steps to use available resources as efficiently as possible.
- 4. The query is executed, and results are produced.

Src: Fundamentals of Data Engineering (Reis & Housley)

Query Optimizer

- Queries can have wildly different execution times, depending on how they're executed. A query optimizer's job is to optimize query performance and minimize costs by breaking the query into appropriate steps in an efficient order.
- Recall The Catalyst Optimizer (Lecture 3)

Optimizing Joining Strategy & Schema

- A single dataset (such as a table or file) is rarely useful on its own; we create value by combining it with other datasets. Joins are one of the most common means of combining datasets and creating new ones.
- A common technique for improving query performance is to prejoin data. If you find that analytics queries are joining the same data repeatedly, it often makes sense to join the data in advance and have queries read from the prejoined version of the data so that you're not repeating computationally intensive work.

Src: Fundamentals of Data Engineering (Reis & Housley)

© Walid Hassan, M.B.A, D.Eng.

Row Explosion

An obscure but frustrating problem is row explosion. This occurs when we have a large number of many-to-many matches, either because of repetition in join keys or as a consequence of join logic. Suppose the join key in table A has the value this repeated five times, and the join key in table B contains this same value repeated 10 times. This leads to a cross-join of these rows: every this row from table A paired with every this row from table B. This creates $5 \times 10 = 50$ rows in the output.

Explain Query

The query optimizer's explain plan will show you how the query optimizer determined its optimum lowest-cost query, the database objects used (tables, indexes, cache, etc.), and various resource consumption and performance statistics in each query stage.

Some databases provide a visual representation of query stages. In contrast, others make the explain plan available via SQL with the EXPLAIN command, which displays the sequence of steps the database will take to execute the query.

Explain Query – Key Items

The following are some areas to monitor:

- Usage of key resources such as disk, memory, and network.
- Data loading time versus processing time.
- Query execution time, number of records, the size of the data scanned, and the quantity of data shuffled.
- Competing queries that might cause resource contention in your database.
- Number of concurrent connections used versus connections available. Oversubscribed concurrent connections can have negative effects on your users who may not be able to connect to the database.

Spark Has the `df.explain()` method

Src: Fundamentals of Data Engineering (Reis & Housley)

© Walid Hassan, M.B.A, D.Eng.

Best Practices:

Avoid full table scans: All queries scan data, but not all scans are created equal. As a rule of thumb, you should query only the data you need. When you run `SELECT *` with no predicates, you're scanning the entire table and retrieving every row and column.

Create & Use indexes on key fields.

Vacuum Dead Records:

As these old records accumulate in the database filesystem, they eventually no longer need to be referenced. You should remove these dead records in a process called vacuuming.

Databricks generally retains data indefinitely until it is manually vacuumed; vacuuming is important to control costs (S3..etc)

Best Practices:

Leverage cached query results: Each time a query is run, this costs you money.

Instead of rerunning the same query on the database repeatedly and incurring massive charges, wouldn't it be nice if the results of the query were stored and available for instant retrieval?

Thankfully, many cloud OLAP databases cache query results.

Note also **that materialized views** provide another form of query caching

Transformations:

- A transformation differs from a query. A query retrieves the data from various sources based on filtering and join logic. A transformation persists the results for consumption by additional transformations or queries. These results may be stored ephemeraly or permanently.
- Besides persistence, a second aspect that differentiates transformations from queries is complexity. You'll likely build complex pipelines that combine data from multiple sources and reuse intermediate results for multiple final outputs. These complex pipelines might normalize, model, aggregate, or featurize data.

Spark Joins:

- **Inner Join:** In the inner join example, only the rows that have matching values in both datasets are returned. We will be joining the cards dataset to the customer data since all cards being associated with a customer performing an inner join is the right approach. To perform an inner join, we will call the `join()` function with the two datasets and specify the join type as inner:

```
customer_cards_df = (  
    cards_df.join(customers_df.  
        on='Cust_ID',  
        how='inner'))  
customer_cards_df.show()
```

Inner Join:

Table 1 

1		
2		

Table 2 

1		
3		
4		

Inner Join 

1				

Src: <https://www.atlassian.com/data/sql/sql-join-types-explained-visually>

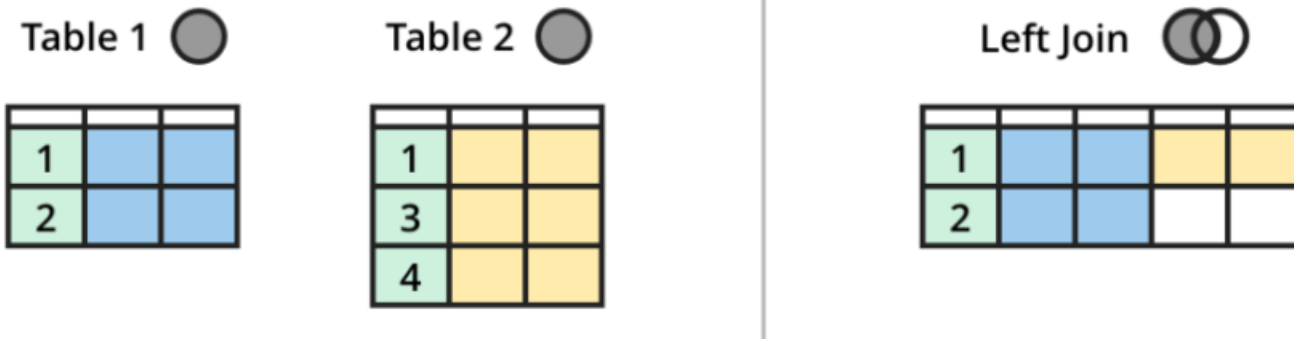
© Walid Hassan, M.B.A, D.Eng.

Spark Joins:

- **Left Outer Join:** the left outer join example, rows in the left datasets are kept in the datasets regardless of their matching with the right dataset.

```
joined_transactions_df = (  
    transactions_df.join(fraud_df,  
        on='Transaction_ID',  
        how='left_outer'))  
joined_transactions_df.show()
```

Left-Outer Join:



Src: <https://www.atlassian.com/data/sql/sql-join-types-explained-visually>

© Walid Hassan, M.B.A, D.Eng.

Spark Joins:

- **Right Outer Join:** To perform a right outer join, we will call the `join()` function with the two datasets and specify the join type as `right_outer`. In the right outer join example, all the rows from the right dataset and matching rows from the left dataset are returned.

```
right_join = df1.join(df2,  
on='Name', how='right_outer')  
right_join.show()
```


Spark Joins:

Full outer join: To perform a full outer join, we will call the `join()` function with the two datasets and specify the join type as `outer`. In the full outer join example, all the rows from both datasets are returned. If there are no matching rows in either dataset, then the values in the corresponding columns will be null.

```
full_join = df1.join(df2,  
on='Name', how='outer')  
full_join.show()
```

Outer Join:

Table 1 

1		
2		

Table 2 

1		
3		
4		

Outer Join 

1				
2				
3				
4				

Src: <https://www.atlassian.com/data/sql/sql-join-types-explained-visually>

© Walid Hassan, M.B.A, D.Eng.

Spark Joins:

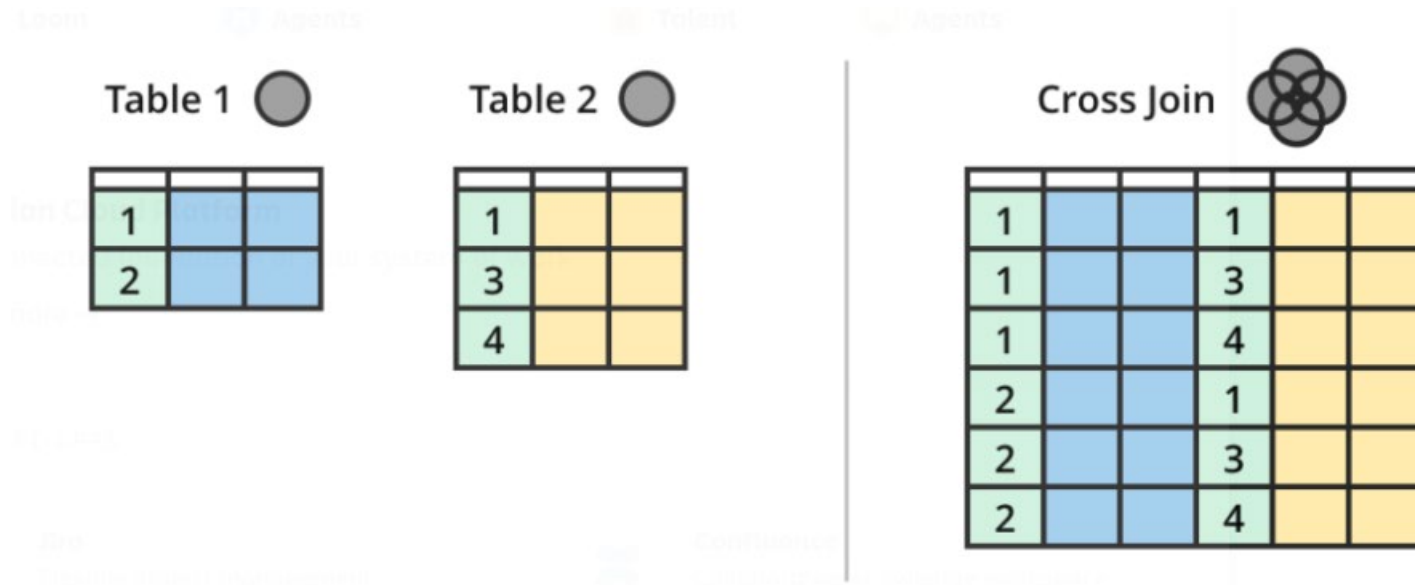
Cross Join: To perform a cross join, we will call the `crossJoin()` function with the two datasets. In the cross join example, all the rows from both datasets are combined in all possible ways. This is also known as a Cartesian product:

```
cross_join = df1.crossJoin(df2)
cross_join.show()
```

Src: Fundamentals of Data Engineering (Reis & Housley)

© Walid Hassan, M.B.A, D.Eng.

Cross Join:



Src: <https://www.atlassian.com/data/sql/sql-join-types-explained-visually>

© Walid Hassan, M.B.A, D.Eng.

Spark Joins:

Multiple Join Conditions: If we want to join datasets on multiple conditions, we can specify them as a list of column names using the `on()` method.

```
multi_join = df1.join(df2,  
on=['Name', 'Gender', 'Age'],  
how='inner')  
multi_join.show()
```

Spark Joins:

Perform joins with complex conditions: Sometimes, we may need to perform joins on complex conditions that involve multiple columns and operators. In such cases, it can be useful to break down the condition into simpler parts and use the & and | operators to combine them

```
joinExpr = (  
    (customer_cards_df["Card_Number"] ==  
    joined_transactions_  
df["Credit_Card_ID"])  
&  
    (joined_transactions_df["Fraud_Flag"].isNull()  
    ))  
customer_with_fraud_df = (  
    Customer_cards_df.join(joined_transactions_df,  
    on=joinExpr,  
    how='inner'))  
customer_with_fraud_df.show()
```

Src: Fundamentals of Data Engineering (Reis & Housley)

© Walid Hassan, M.B.A, D.Eng.

Spark Joins:

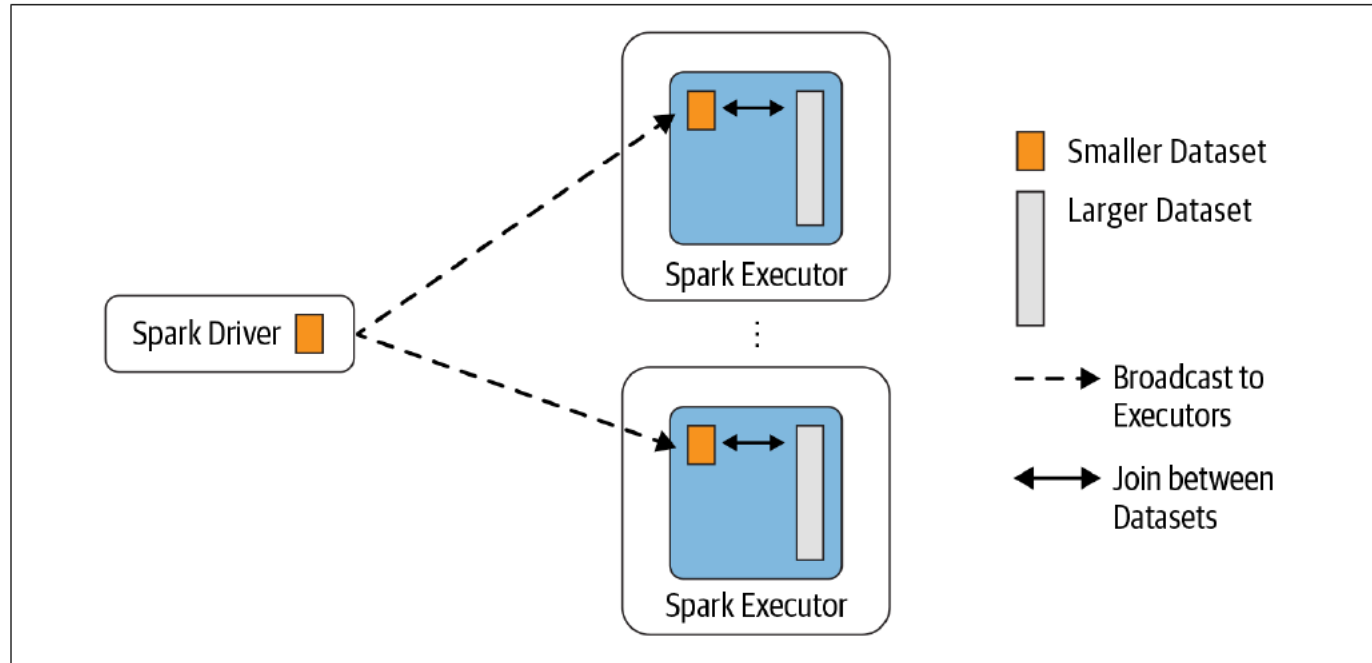
- Join operations are a common type of transformation in big data analytics in which two data sets, in the form of tables or DataFrames, are merged over a common matching key.
- At the heart of these transformations is how Spark computes what data to produce, what keys and associated data to write to the disk, and how to transfer those keys and data to nodes as part of operations like `groupBy()`, `join()`, `agg()`, `sortBy()`, and `reduceByKey()`. This movement is commonly referred to as the shuffle.
- We will focus on broadcast hash join (BHJ), shuffle sort merge join (SMJ).

Broadcast Hash Join

- Broadcast Hash Join Also known as a map-side-only join, the broadcast hash join is employed when two data sets, one small (fitting in the driver's and executor's memory) and another large enough to ideally be spared from movement, need to be joined over certain conditions or columns. Using a Spark broadcast variable, the smaller data set is broadcasted by the driver to all Spark executors.
- By default Spark will use a broadcast join if the smaller data set is less than 10 MB.
- The BHJ is the easiest and fastest join Spark offers, since it does not involve any shuffle of the data set; all the data is available locally to the executor after a broadcast. You just have to be sure that you have enough memory both on the Spark driver's and the executors' side to hold the smaller data set in memory

Src: Learning Spark, Damji et al..

Broadcast Hash Join



- `joinedDF.explain(mode)`. The modes include 'simple', 'extended', 'codegen', 'cost', and
- 'formatted'.

Src: Learning Spark, Damji et al..

© Walid Hassan, M.B.A, D.Eng.

Broadcast Hash Join

When to use a broadcast hash join:

Use this type of join under the following conditions for maximum benefit:

- When each key within the smaller and larger data sets is hashed to the same partition by Spark
- When one data set is much smaller than the other (and within the default config of 10 MB, or more if you have sufficient memory)
- When you only want to perform an equi-join, to combine two data sets based on matching unsorted keys
- When you are not worried by excessive network bandwidth usage or OOM errors, because the smaller data set will be broadcast to all Spark executors

Src: Learning Spark, Damji et al..

© Walid Hassan, M.B.A, D.Eng.

Shuffle Sort Merge Join (SMJ)

- The sort-merge algorithm is an efficient way to merge two large data sets over a common key that is sortable, unique, and can be assigned to or stored in the same partition— that is, two data sets with a common hashable key that end up being on the same partition.
- From Spark's perspective, this means that all rows within each data set with the same key are hashed on the same partition on the same executor. Obviously, this means data has to be colocated or exchanged between executors. this join scheme has two phases: a sort phase followed by a merge phase.
- The sort phase sorts each data set by its desired join key; the merge phase iterates over each key in the row from each data set and merges the rows if the two keys match.

Src: Learning Spark, Damji et al..

© Walid Hassan, M.B.A, D.Eng.

Shuffle Sort Merge Join (SMJ)

Use this type of join under the following conditions for maximum benefit:

- When each key within two large data sets can be sorted and hashed to the same partition by Spark
- When you want to perform only equi-joins to combine two data sets based on matching sorted keys
- When you want to prevent Exchange and Sort operations to save large shuffles across the network

Src: Learning Spark, Damji et al..

© Walid Hassan, M.B.A, D.Eng.



Hands On.....

Common DataFrames and Spark SQL Ops

- Aggregate functions
- Collection functions
- Datetime functions
- Math functions
- Miscellaneous functions
- Non-aggregate functions
- Sorting functions
- String functions
- UDF functions
- Window functions

Src: Learning Spark, Damji et al..

© Walid Hassan, M.B.A, D.Eng.

Common DataFrames and Spark SQL Ops

https://docs.databricks.com/en/sql/language-manual/functions/array_distinct.html

<https://docs.databricks.com/en/udf/python.html>

Src: Learning Spark, Damji et al..

© Walid Hassan, M.B.A, D.Eng.

Key Functions

Transform:

```
transform(values, value -> lambda expression)
```

The `transform()` function takes an array (`values`) and an anonymous function (`lambda expression`) as input. The function transparently creates a new array by applying the anonymous function to each element, and then assigning the result to the output array.

Src: Learning Spark, Damji et al..

© Walid Hassan, M.B.A, D.Eng.

Key Functions

`transform(): transform(array<T>, function<T, U>): array<U>`

In Python

```
from pyspark.sql.types import *
```

```
schema = StructType([StructField("celsius", ArrayType(IntegerType()))])
```

```
t_list = [[35, 36, 32, 30, 40, 42, 38]], [[31, 32, 34, 55, 56]]
```

```
t_c = spark.createDataFrame(t_list, schema)
```

```
t_c.createOrReplaceTempView("tC")
```

```
// Calculate Fahrenheit from Celsius for an array of temperatures
```

```
spark.sql(""" SELECT celsius,transform(celsius, t -> ((t * 9) div 5) + 32) as Fahrenheit FROM tC  
""").show()
```

Src: Learning Spark, Damji et al..

© Walid Hassan, M.B.A, D.Eng.

Aggregation

Perform aggregations: In Apache Spark, the `groupBy` operation is used to group data in a distributed collection (`DataFrame`) based on one or more keys. It allows you to perform aggregations or transformations on the grouped data. The `groupBy` operation requires specifying one or more keys based on which the grouping will occur. These keys can be column names in a `DataFrame`.

```
# Group the data by a column
grouped_df =
df.groupBy("country")
```

The **`groupBy` operation in Spark is a transformation**, which means it is **lazily** evaluated

Aggregation

After the `groupBy` operation, you can apply various aggregation or transformation functions to the grouped data. These functions include `count`, `sum`, `avg`, `min`, `max`, and so on.

```
# Count the number of rows in
each group
count_df = grouped_df.count()
count_df.show()
```

You can also apply custom aggregation functions using the `agg` function

```
# Apply custom aggregation using
max
max_release_df =
grouped_df.agg(max(col("date_added")))
max_release_df.show()
```

Key Functions

Filter(): filter(array<T>, function<T, Boolean>): array<T>

The filter() function produces an array consisting of only the elements of the input array for which the Boolean function is true:

```
// In Scala/Python
```

```
// Filter temperatures > 38C for array of temperatures
```

```
spark.sql(""" SELECT celsius, filter(celsius, t -> t > 38) as high FROM tC""").show()
```

Src: Learning Spark, Damji et al..

© Walid Hassan, M.B.A, D.Eng.

Key Functions

Combine multiple conditions: We can also combine multiple conditions using the & (and) and | (or) operators. In this example, we will filter the DataFrame to only include titles from the United States and whose release_year is greater than 2020.

```
filtered_df = ( df.filter( (col("country") == "United States") & (col("release_year") > 2020)))  
  
filtered_df.show()
```

Filter based on a list of values:

```
filtered_df = ( df.filter( col("country").isin(["United States", "United Kingdom", "India"])))  
  
filtered_df.show()
```

Key Functions

Filtering on string:

To filter on string substrings in Apache Spark, you can use the `like` or `rlike` functions. The `like` function allows you to filter based on a pattern match using wildcard characters, while the `rlike` function allows you to filter based on a regular expression match

filter the DataFrame based on a substring match

```
filtered_df = df.filter(col("listed_in").like("%Crime%"))
```

filter the DataFrame based on a regular expression match

```
filtered_df = df.filter(col("listed_in").rlike("(Crime|Thrillers)"))
```

Key Functions

Filtering on data ranges:

To filter on data ranges with Apache Spark, you can use comparison operators such as `>`, `>=`, `<`, `<=`, or `between` to compare column values against specific ranges.

filter the DataFrame based on a date range

```
filtered_df = df.filter((col("date_added") >= "2021-02-01") &  
(col("date_added") <= "2021-03-01"))
```

Src: Data Engineering With Databricks cookbook, Pulkit Chadha

© Walid Hassan, M.B.A, D.Eng.

Key Functions

Filtering on arrays: you can use the `array_contains` function or use the `explode` function to convert the array column into multiple rows and then apply filter operations.

```
from pyspark.sql.functions import array_contains, col
```

```
# Read parquet file into a DataFrame
```

```
df_recipes = (spark.read .format("parquet") .load("../data/recipes.parquet"))
```

filter the DataFrame based on a value in the array column

```
filtered_df = df_recipes.filter(array_contains(col("RecipeIngredientParts"),  
"apple"))
```

```
# display the filtered DataFrame
```

```
filtered_df.show()
```

Src: Data Engineering With Databricks cookbook, Pulkit Chadha

© Walid Hassan, M.B.A, D.Eng.

Key Functions

`exists()`

`exists(array<T>, function<T, V, Boolean>): Boolean`

The `exists()` function returns true if the Boolean function holds for any element in the input array:

```
// In Scala/Python
```

```
// Is there a temperature of 38C in the array of temperatures
```

```
spark.sql(""" SELECT celsius, exists(celsius, t -> t = 38) as threshold FROM tC """).show()
```

Src: Learning Spark, Damji et al..

© Walid Hassan, M.B.A, D.Eng.

Key Functions

`reduce()` : `reduce(array<T>, B, function<B, T, B>, function<B, R>)` The `reduce()` function reduces the elements of the array to a single value by merging the elements into a buffer B using `function<B, T, B>` and applying a finishing function `<B, R>` on the final buffer

// In Scala/Python

// Calculate average temperature and convert to F

```
spark.sql(""" SELECT celsius, reduce( celsius, 0, (t, acc) -> t + acc, acc -> (acc div size(celsius) * 9  
div 5) + 32 ) as avgFahrenheit FROM tC """).show()
```

Src: Learning Spark, Damji et al..

© Walid Hassan, M.B.A, D.Eng.

Modifications

Another common operation is to perform modifications to the DataFrame. While DataFrames themselves are immutable, you can modify them through operations that create new, different DataFrames, with different columns.

Adding new columns

To add a new column to the foo DataFrame, use the withColumn() method:

In Python

```
from pyspark.sql.functions import expr
```

```
foo2 = (foo.withColumn( "status",
```

```
expr("CASE WHEN delay <= 10 THEN 'On-time' ELSE 'Delayed' END")
```

```
))
```

Src: Learning Spark, Damji et al..

© Walid Hassan, M.B.A, D.Eng.

Modifications

Dropping columns

To drop a column, use the `drop()` method. For example, let's remove the `delay` column as we now have a `status` column, added in the previous section:

In Python

```
foo3 = foo2.drop("delay")
```

```
foo3.show()
```

Src: Learning Spark, Damji et al..

© Walid Hassan, M.B.A, D.Eng.

Modifications

Renaming columns

You can rename a column using the `rename()` method:

In Python

```
foo4 = foo3.withColumnRenamed("status", "flight_status")
```

Pivoting:

When working with your data, you might need to swap the columns for the rows – a process known as pivoting. This can be done using either:

- SQL with the PIVOT clause.
- Python with the PySpark DataFrame `pivot()` API.

User Defined Functions (UDFs)

While Apache Spark has a plethora of built-in functions, the flexibility of Spark allows for data engineers and data scientists to define their own functions too. These are known as user-defined functions.

```
from pyspark.sql.types import LongType
```

```
# Create cubed function
```

```
def cubed(s):
```

```
    return s * s * s
```

```
# Register UDF
```

```
spark.udf.register("cubed", cubed, LongType())
```

```
spark.sql("SELECT id, cubed(id) AS id_cubed FROM udf_test").show()
```

id	id_cubed
1	1
2	8
3	27
4	64
5	125
6	216
7	343
8	512

Src: Learning Spark, Damji et al..

User Defined Functions (UDFs)

Spark UDFs allow you to apply custom functions to your data. Standard UDFs execute operations row by row and do not vectorize computations, which can lead to performance overhead. However, Pandas UDFs (also known as vectorized UDFs) process data in batches using Apache Arrow, enabling vectorized execution for better performance..

```
# In Python
# Import pandas
import pandas as pd
# Import various pyspark SQL functions including pandas_udf
from pyspark.sql.functions import col, pandas_udf
from pyspark.sql.types import LongType
# Declare the cubed function
def cubed(a: pd.Series) -> pd.Series:
    return a * a * a
# Create the pandas UDF for the cubed function
cubed_udf = pandas_udf(cubed, returnType=LongType())
```

Src: Learning Spark, Damji et al..

Windowing Functions

Windowing: A window function uses values from the rows in a window (a range of input rows) to return a set of values, typically in the form of another row. With window functions, it is possible to operate on a group of rows while still returning a single value for every input row.

	SQL	DataFrame API
Ranking functions	rank()	rank()
	dense_rank()	denseRank()
	percent_rank()	percentRank()
	ntile()	ntile()
	row_number()	rowNumber()
Analytic functions	cume_dist()	cumeDist()
	first_value()	firstValue()
	last_value()	lastValue()
	lag()	lag()
	lead()	lead()

Src: Learning Spark, Damji et al..

Materialized Views, Federation, and Query Virtualization

These techniques that virtualize query results by presenting them as table-like objects. These techniques can become part of a transformation pipeline or sit right before end-user data consumption.

Views:

A view is a database object that we can select from just like any other table. In practice, a view is just a query that references other tables. When we select from a view, that database creates a new query that combines the view subquery with our query. The query optimizer then optimizes and runs the full query.

Materialized Views, Federation, and Query Virtualization

Views play a variety of roles in a database : **First**, views can serve a security role. For example, views can select only specific columns and filter rows, thus providing restricted data access.

Various views can be created for job roles depending on user data access.

Second, a view might be used to provide a current deduplicated picture of data. If we're using an insert-only pattern, a view may be used to return a deduplicated version of a table showing only the latest version of each record.

Third, views can be used to present common data access patterns. Suppose that marketing analysts must frequently run a query that joins five tables. We could create a view that joins together these five tables into a wide table. Analysts can then write queries that filter and aggregate on top of this view.

Src: Fundamentals of Data Engineering (Reis & Housley)

© Walid Hassan, M.B.A, D.Eng.

Materialized Views, Federation, and Query Virtualization

Materialized views: A materialized view does some or all of the view computation in advance. In our example, a materialized view might save the five table join results every time a change occurs in the source tables. Then, when a user references the view, they're querying from the prejoined data. A materialized view is a de facto transformation step, but the database manages execution for convenience.

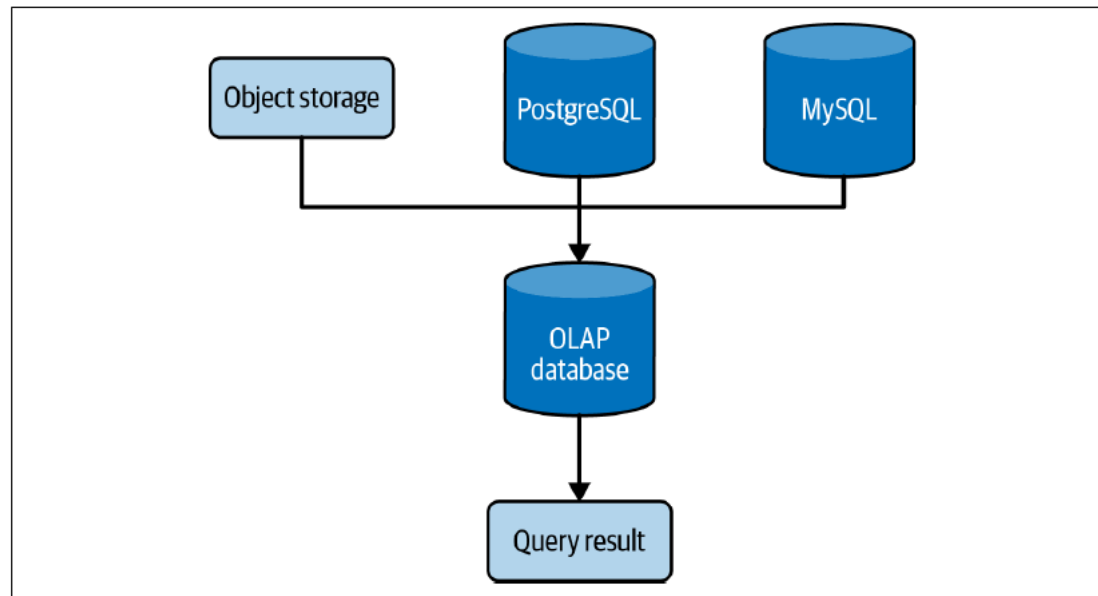
Materialized views may also serve a significant query optimization role depending on the database, even for queries that don't directly reference them. Many query optimizers can identify queries that “look like” a materialized view. An analyst may run a query that uses a filter that appears in a materialized view. The optimizer will rewrite the query to select from the precomputed results.

Materialized Views, Federation, and Query Virtualization

In general, materialized views do not allow for composition—that is, a materialized view cannot select from another materialized view. Databricks has introduced the notion of live tables. Each table is updated as data arrives from sources. Data flows down to subsequent tables asynchronously.

Materialized Views, Federation, and Query Virtualization

Federated queries are a database feature that allows an OLAP database to select from an external data source, such as object storage or RDBMS. E.g. An OLAP database issues a federated query that gets data from object storage, MySQL, and PostgreSQL and returns a query result with the combined data



Src: Fundamentals of Data Engineering (Reis & Housley)

© Walid Hassan, M.B.A, D.Eng.

Materialized Views, Federation, and Query Virtualization

- Data virtualization is closely related to federated queries, but this typically entails a data processing and query system that doesn't store data internally.
- Any query/processing engine that supports external tables can serve as a data virtualization engine. The most significant considerations with data virtualization are supported external sources and performance.

Materialized Views, Federation, and Query Virtualization

Data virtualization is a good solution for organizations with data stored across various data sources. This fits closely with the notion of a data mesh, wherein small teams are responsible for preparing their data for analytics and sharing it with the rest of the company; virtualization can serve as a critical access layer for practical sharing

Src: Fundamentals of Data Engineering (Reis & Housley)

© Walid Hassan, M.B.A, D.Eng.

Creating Views

- Spark can create views on top of existing tables. Views can be global (visible across all SparkSessions on a given cluster) or session-scoped (visible only to a single SparkSession), and they are temporary: they disappear after your Spark application terminates.
- Creating views has a similar syntax to creating tables within a database. Once you create a view, you can query it as you would a table. The difference between a view and a table is that views don't actually hold the data; tables persist after your Spark application terminates, but views disappear.
- You can create a view from an existing table using SQL/Python.

Creating Views

In Python

```
df_sfo = spark.sql("SELECT date, delay, origin, destination FROM  
us_delay_flights_tbl WHERE origin = 'SFO'")
```

```
df_jfk = spark.sql("SELECT date, delay, origin, destination FROM  
us_delay_flights_tbl WHERE origin = 'JFK'")
```

Create a temporary and global temporary view

```
df_sfo.createOrReplaceGlobalTempView("us_origin_airport_SFO_global_tmp_view")
```

```
df_jfk.createOrReplaceTempView("us_origin_airport_JFK_tmp_view")
```

Once you've created these views, you can issue queries against them just as you would against a table. Keep in mind that when accessing a global temporary view you must use the prefix `global_temp.<view_name>`

By contrast, you can access the normal temporary view without the `global_temp` prefix.

Viewing Metadata

Spark manages the metadata associated with each managed or unmanaged table. This is captured in the Catalog, a high-level abstraction in Spark SQL for storing metadata.

you can access all the stored metadata through methods like these:

```
// In Scala/Python  
spark.catalog.listDatabases()  
spark.catalog.listTables()  
spark.catalog.listColumns("us_delay_flights_tbl")
```

Caching and Persistence of Data

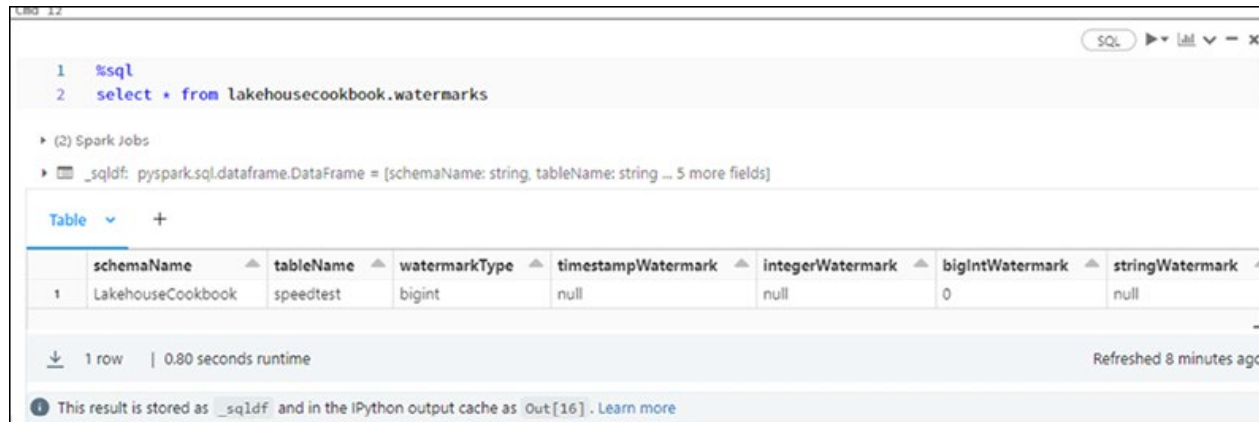
- What is the difference between caching and persistence? In Spark they are synonymous.
- Two API calls, `cache()` and `persist()`, offer these capabilities. The latter provides more control over how and where your data is stored—in memory and on disk, serialized and unserialized.
- **DataFrame.cache()**
 - `cache()` will store as many of the partitions read in memory across Spark executors as memory allows.
 - a DataFrame may be fractionally cached, partitions cannot be fractionally cached (e.g., if you have 8 partitions but only 4.5 partitions can fit in memory, only 4 will be cached). However, if not all your partitions are cached, when you want to access the data again, the partitions that are not cached will have to be recomputed, slowing down your Spark job.
- Remember that Spark uses Lazy Evaluation: i.e. Spark defers computation until an action (like `count()`) is called.
- When `df.cache()` is called, Spark marks the DataFrame for caching but doesn't immediately store it.

Src: Fundamentals of Data Engineering (Reis & Housley)

© Walid Hassan, M.B.A, D.Eng.

Loading from Bronze → Silver

- Remember, it is most efficient to do incremental data loads.....
- For that we use a “watermark” which is nothing more than the “last update” key (most cases it is a timestamp).
- The only determining characteristic of a watermark is that it can be sorted.
- We also need to know the maximum value of the records we have processed, as it will become the starting point for our next refinement cycle.



The screenshot shows a Databricks SQL interface. At the top, there is a code editor with the following SQL query:

```
1 %sql
2 select * from lakehousecookbook.watermarks
```

Below the code editor, the execution status is shown: (2) Spark Jobs. A message indicates the query was successful: `_sqlidf: pyspark.sql.dataframe.DataFrame = [schemaName: string, tableName: string ... 5 more fields]`.

The results are displayed in a table with 8 columns: `schemaName`, `tableName`, `watermarkType`, `timestampWatermark`, `integerWatermark`, `bigIntWatermark`, and `stringWatermark`. There is one row of data:

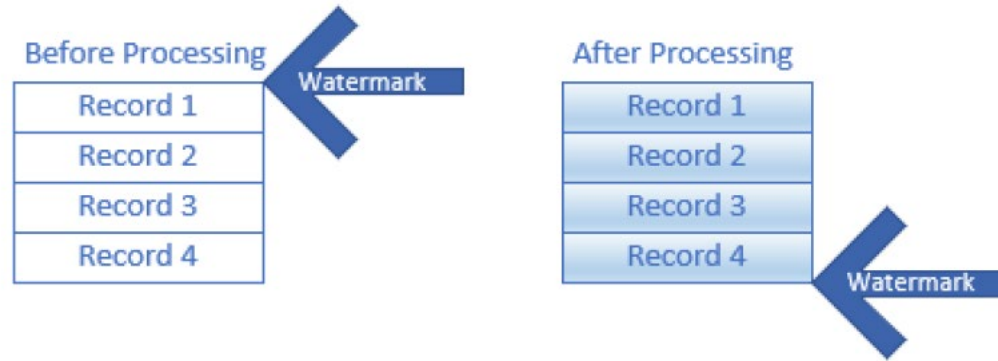
	schemaName	tableName	watermarkType	timestampWatermark	integerWatermark	bigIntWatermark	stringWatermark
1	LakehouseCookbook	speedtest	bigint	null	null	0	null

At the bottom of the table, it indicates "1 row" and "0.80 seconds runtime". A note at the bottom states: "This result is stored as `_sqlidf` and in the IPython output cache as `Out[16]`. Learn more". The interface also shows a "Refreshed 8 minutes ago" status.

Src: Databricks Lakehouse Platform Cookbook (Dr. Alan Dennis)

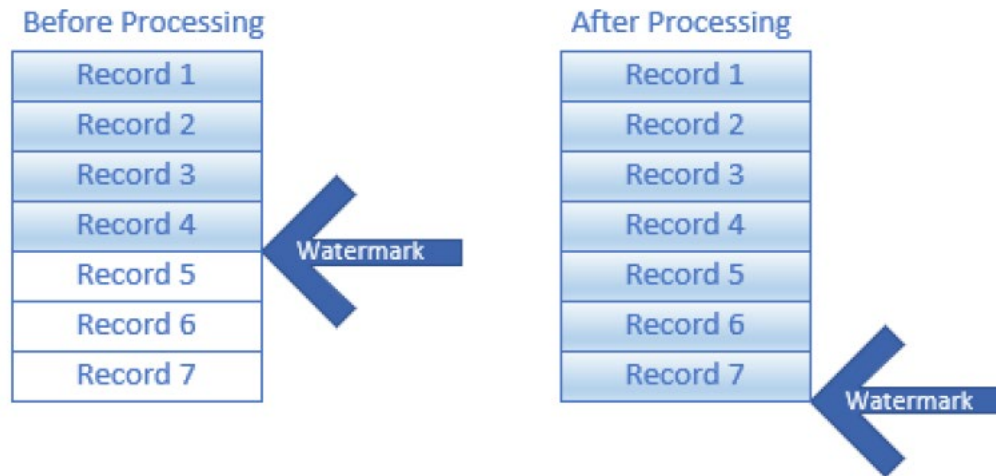
© Walid Hassan, M.B.A, D.Eng.

Loading from Bronze → Silver



- As an example, consider a table with four records. Initially, the Watermark for that table should be the lowest value for the data type associated with the Watermark column.
- Once the first four records are processed, the watermark is updated to reflect the Watermark value of the last processed record. When new records arrive, that persisted Watermark value is the starting point for processing.

Loading from Bronze → Silver



- It reduces the amount of data being processed during each cycle. This, in turn, reduces the time needed for processing, along with reducing the cost to do that processing

References

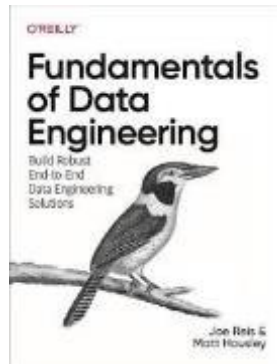
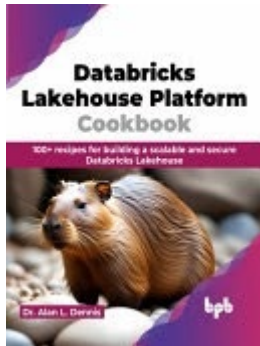
In addition to the references in each slide the below are leveraged throughout this course.

Gareth James, D. W. (2023). An Introduction to Statistical Learning: with Applications in Python. Springer.

(Jules S. Damji, July 2020), Learning Spark Lightning Fast Data Analytics , 2020 (2nd Edition).

Databricks Lakehouse Platform Cookbook, Dennis, Dr. Alan L.

Fundamentals of Data Engineering, O'Reilly (J. Reis, M. Housley)



O'REILLY®

Learning Spark

Lightning-Fast Data Analytics

2nd Edition
Covers
Apache Spark 3.0



Jules S. Damji,
Brooke Wenig,
Tathagata Das
& Denny Lee
Foreword by Matei Zaharia

© Walid Hassan, M.B.A, D.Eng.

THE GEORGE
WASHINGTON
UNIVERSITY
WASHINGTON, DC